CS 498 · Machine Learning Systems

LLM Quantization

Make large models smaller without losing the signal

Quantization fundamentals · SmoothQuant · AWQ discussion

Fan Lai | University of Illinois Urbana-Champaign

Fall 2026 · Materials with thanks to Minjia Zhang, Arvind Krishnamurthy, and colleagues

Recall: adaptive KV cache reduced memory by keeping fewer tokens

Last lecture: model state has more than one source of memory pressure.

KV cache retain fewer tokens	**Weights use fewer bits**	**Activations use fewer bits**
H2O / StreamingLLM	Today's quantization	SmoothQuant: W8A8

Takeaways Quantization complements KV-cache management. It does not replace it.

Today’s lecture

How can we lower inference cost while preserving model quality?

01	**Motivation**	Why precision affects memory and compute
02	**Quantization basics**	Rounding, scales, and granularity
03	**SmoothQuant**	Tame activation outliers without changing outputs
04	**Evaluate + discuss Murakkab**	Accuracy and latency tradeoff in agent serving

Learning objectives

By the end of today, we should be able to…

Explain	the scale / zero-point mapping and its quantization error.
Apply	symmetric quantization and compare per-tensor with per-channel scales.
Analyze	how SmoothQuant migrates difficulty between X (activation/input) and W (weight).
Evaluate	W8A8 versus weight-only quantization using accuracy and runtime metrics.

Why quantize an LLM?

A 7B-parameter model has substantial weight storage even before KV cache and activations.



Illustrative decimal GB (7×10^9 parameters; storage only).

Takeaways Lower precision saves bytes; speedup also depends on hardware kernels and conversion overhead.

What exactly gets quantized?

W16A16	W8A8	W4A16
Full / half precision	**INT8 matrix multiplies**	**Weight-only low-bit**
FP16 weights + FP16 activations	INT8 weights + INT8 activations	INT4 weights + FP16 activations

Quantization format is not a performance guarantee: execution kernels, packing, and memory traffic matter.

Takeaways SmoothQuant targets W8A8, while AWQ targets low-bit weight-only quantization.

Uniform affine quantization: scale and zero point

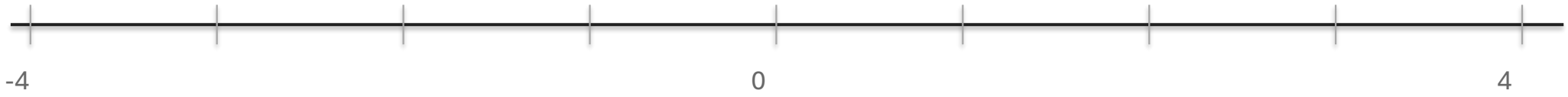
Map real-valued values to a finite grid of integers.

$$q = \text{clip}(\text{round}(x / S) + Z)$$

$$\hat{x} = S\,(q - Z)$$

$S > 0$: scale / quantization step size

Z: zero point; q is bounded by bit width



Takeaways Quantization incurs rounding and possible clipping errors. A larger S gives coarser steps.

Round-to-nearest: a worked INT4 example

Illustrative symmetric INT4, signed range −7 … +7

max |x| = 1.4 → S = 1.4 / 7 = 0.2

x = 0.68 → q = round(0.68 / 0.2) = 3

x̂ = 3 × 0.2 = 0.60 → |error| = 0.08

x̂ = S (q − Z)

q = clip(round(x / S) + Z)



Integer grid after rounding

Takeaways Even without clipping, rounding to a coarse grid changes the value.

Symmetric versus asymmetric quantization

Symmetric (signed INT8)

Z = 0

$S \approx \max|x| / 127$
Range: −127 … +127

Asymmetric (unsigned INT8)

Z can be nonzero

$S = (x_{max} - x_{min}) / 255$
$Z \approx \text{round}(-x_{min} / S)$

Both need a scale and have rounding error. Hardware support and tensor distribution guide the choice.

Takeaways Use symmetric scales for simplicity. Asymmetric scales can represent skewed ranges more efficiently.

Quantization granularity: one scale or many?

Per tensor



one S for a whole tensor

Per channel



different S by channel

Per token



different S by token

Group-wise



one S per group of weights

Takeaways Finer granularity can limit outlier impact, at the cost of more scales and execution complexity.

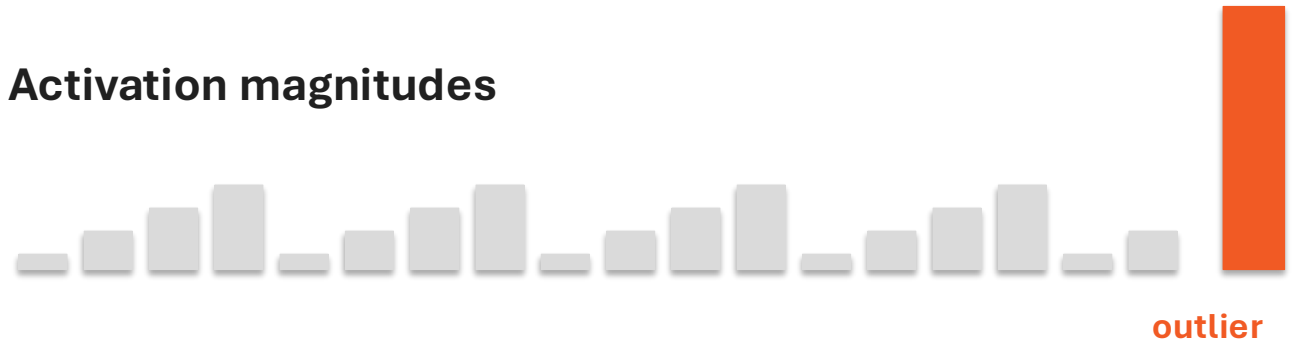
Choosing granularity is a systems decision

Scheme	How scales are shared	Typical tradeoff
Per-tensor	one across tensor	simple, sensitive to outliers
Per-channel	one per weight channel	good precision
Per-token	one per activation token	adapts to token variation
Group-wise	one per small block	popular for low-bit weights

Takeaways Per-tensor W8A8 needs careful handling of activation outliers. Granularity alone is not enough.

LLM quantization: activations are the hard part

Uniform quantization assumes most values fit reasonably well into the same range.



A rare large value expands the scale for everyone else.

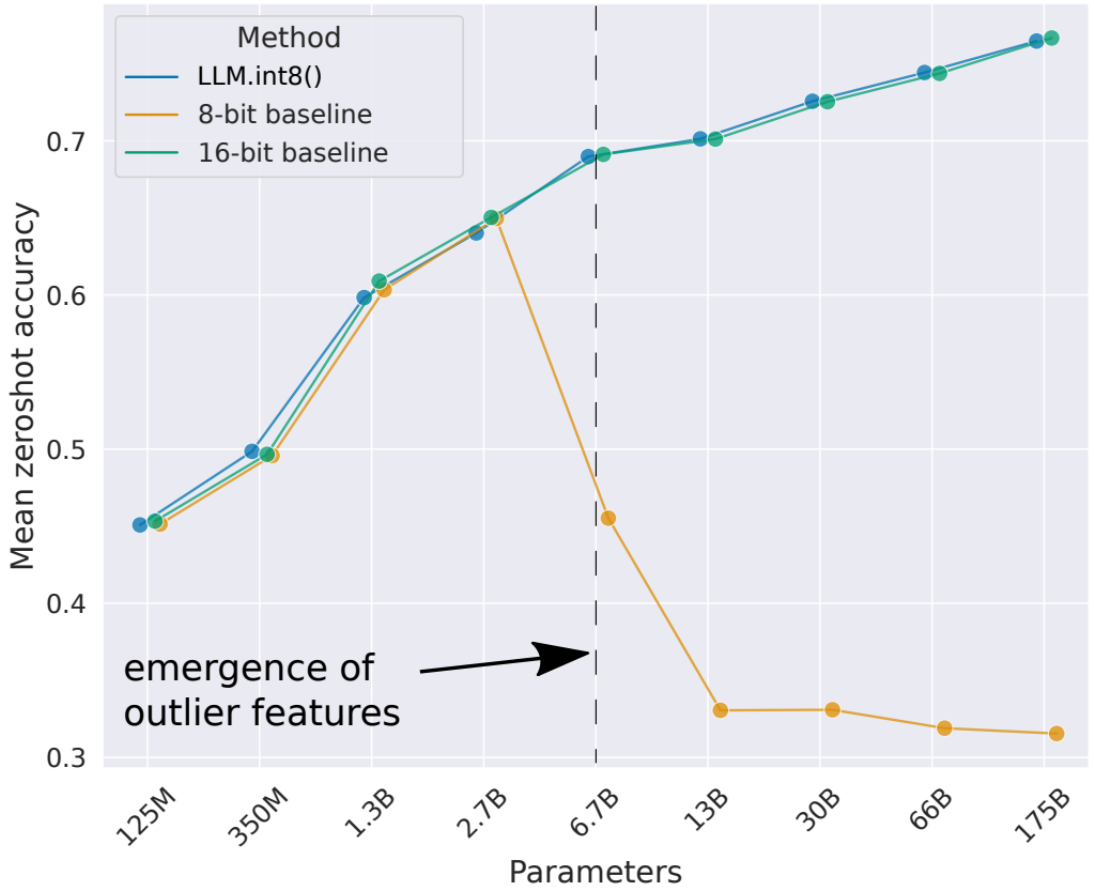
Most small values then use only a tiny fraction of available integer levels.

Takeaways An outlier may dominate the quantization scale while carrying little probability mass.

Evidence: simple W8A8 can lose accuracy

From LLM.int8()

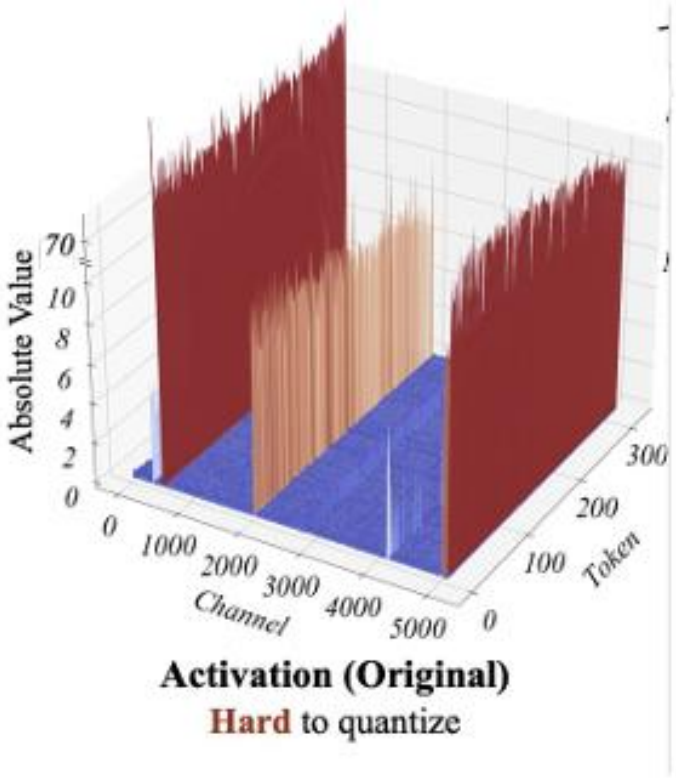
Quantizing activations can cause severe degradation as models grow.

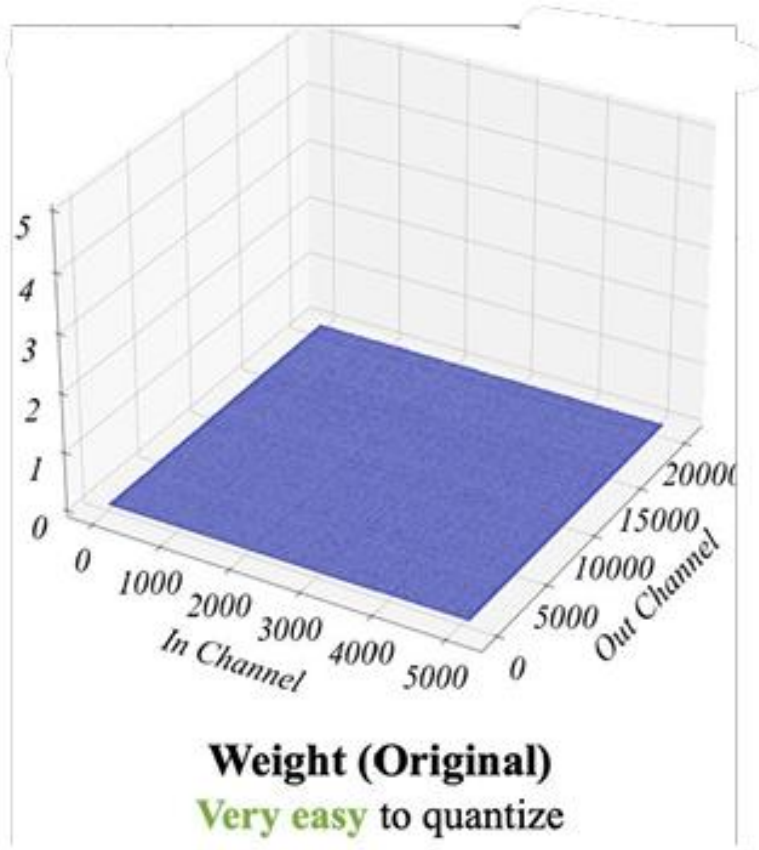


Takeaways Activation outliers motivate methods beyond naive per-tensor INT8.

Source: Dettmers et al., LLM.int8() (2022/2023); figure from supplied lecture.

The activation and weight distributions look different

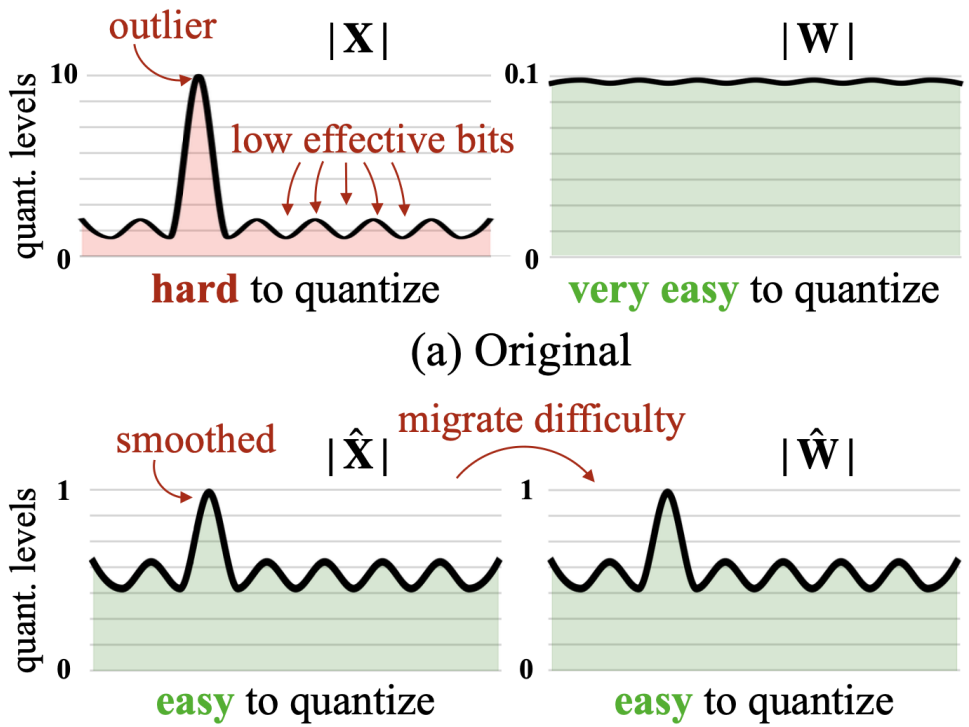




Activations: strong outliers | Weights: comparatively smooth

Source: Xiao et al., SmoothQuant, ICML 2023; illustration from supplied lecture.

SmoothQuant: migrate difficulty to the weights



For a positive diagonal scaling matrix D = diag(s):

$$\mathbf{Y} = \mathbf{XW} = (\mathbf{X}\,\mathbf{D}^{-1})(\mathbf{D}\,\mathbf{W})$$

Smoothed activations

Compensated weights

e.g., $\mathbf{Y} = \mathbf{XW} = (0.01\mathbf{X})(100\mathbf{W})$

The factors D^{-1} and D cancel in full precision.

Source: Xiao et al., "SmoothQuant," ICML 2023; conceptual figure reused from supplied lecture.

SmoothQuant: choose one scale per input channel

Let $a_j = \max_t |X_{tj}|$ and $w_j = \max_k |W_{jk}|$ from calibration and weights.

$$s_j = (a_j)^{\alpha} / (w_j)^{(1-\alpha)}$$

For a positive diagonal scaling matrix D = diag(s):

$$Y = XW = (X D^{-1})(D W)$$

Smoothed activations

Compensated weights

α = 0: favor weight equalization

α = 1: favor activation smoothing

Intermediate α balances the quantization difficulty for the two operands.

Takeaways α is not an optimization learning rate; it controls how aggressively activation range moves into weights.

SmoothQuant channel scaling rule, Xiao et al., ICML 2023.

A two-channel smoothing example

Let $a_j = \max_t |X_{tj}|$ and $w_j = \max_k |W_{jk}|$ from calibration and weights.

$$s_j = (a_j)^{\alpha} / (w_j)^{(1-\alpha)}$$

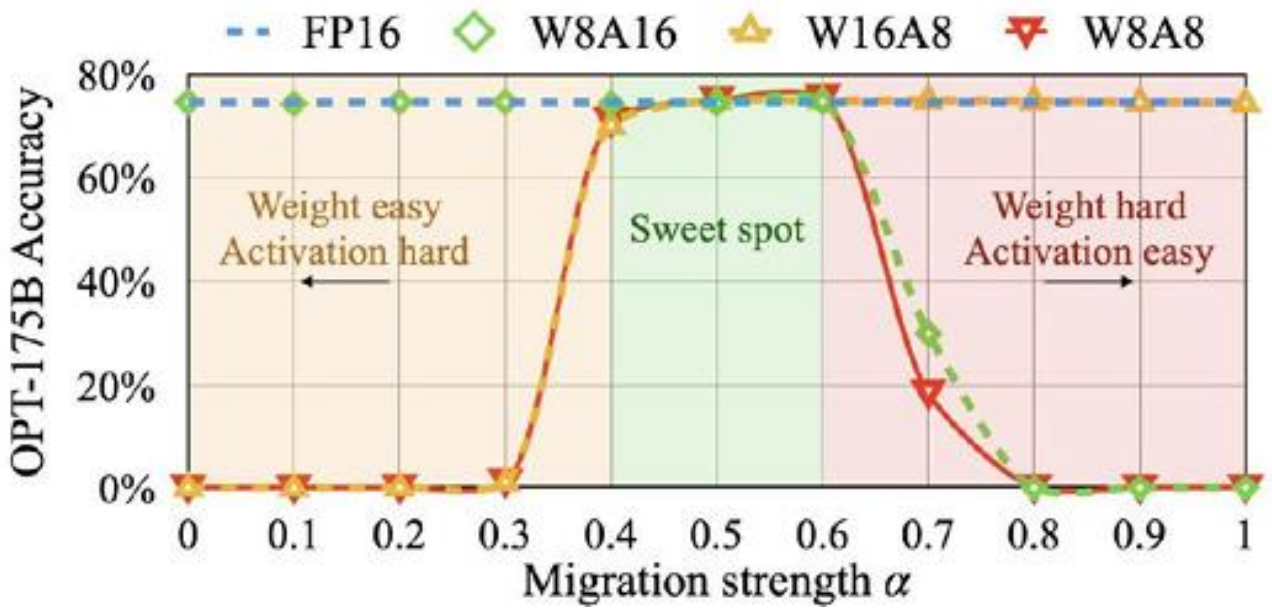
Take $\alpha = 0.5$ and two activation / weight channels.

Channel	max $\lvert X_j \rvert$	max $\lvert W_j \rvert$	s_j	After scaling: X / s ; sW
1	2	4	$\sqrt{0.5} \approx 0.71$	2.82 ; 2.84
2	16	1	4	4 ; 4

Magnitudes are now more balanced in both X' and W'.

Choosing α is a tradeoff

Increasing α makes activations easier to quantize but can make weights harder.



Paper curve: OPT-175B accuracy changes with migration strength and quantization mode.

Takeaways Use representative calibration data and validate α against model accuracy and runtime.

Source: Xiao et al., SmoothQuant, ICML 2023; α sweep from supplied lecture.

How SmoothQuant fits into an inference stack

Offline preparation

Calibrate X → **Compute s_j** → **Rescale weights**

Online serving

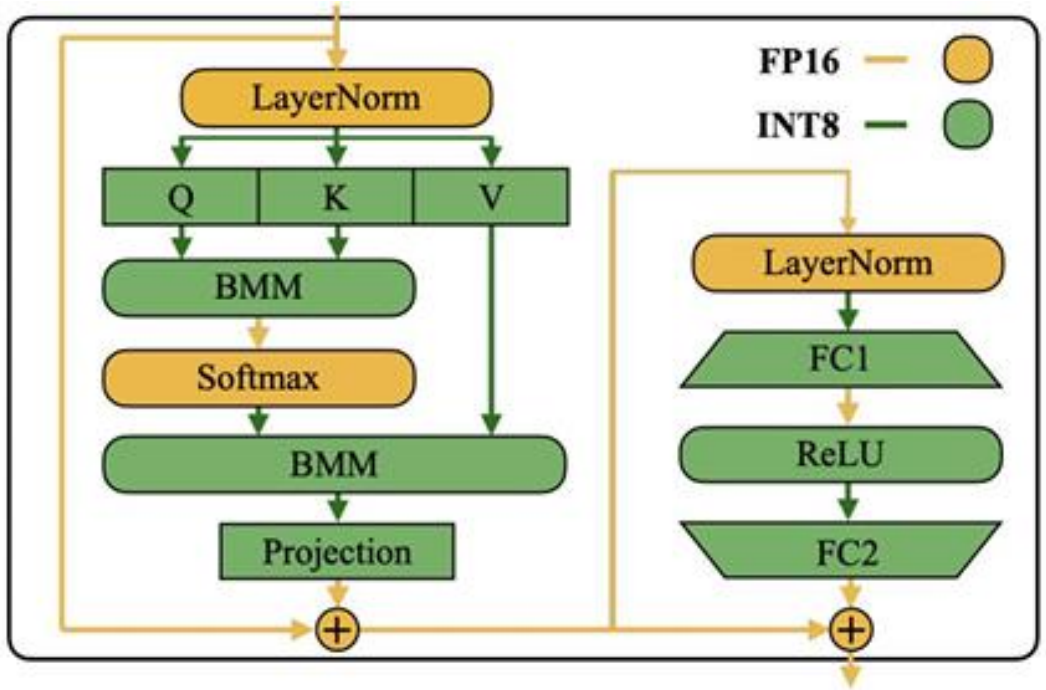
Quantize input → **INT8 GEMM** → **Dequant / other ops**

FP16 / higher precision for selected lightweight ops

Takeaways The rescaling can be absorbed into existing layers, limiting online overhead.

Hardware perspective: quantize the expensive operators

SmoothQuant targets matrix multiplications, so not every operation must become INT8.

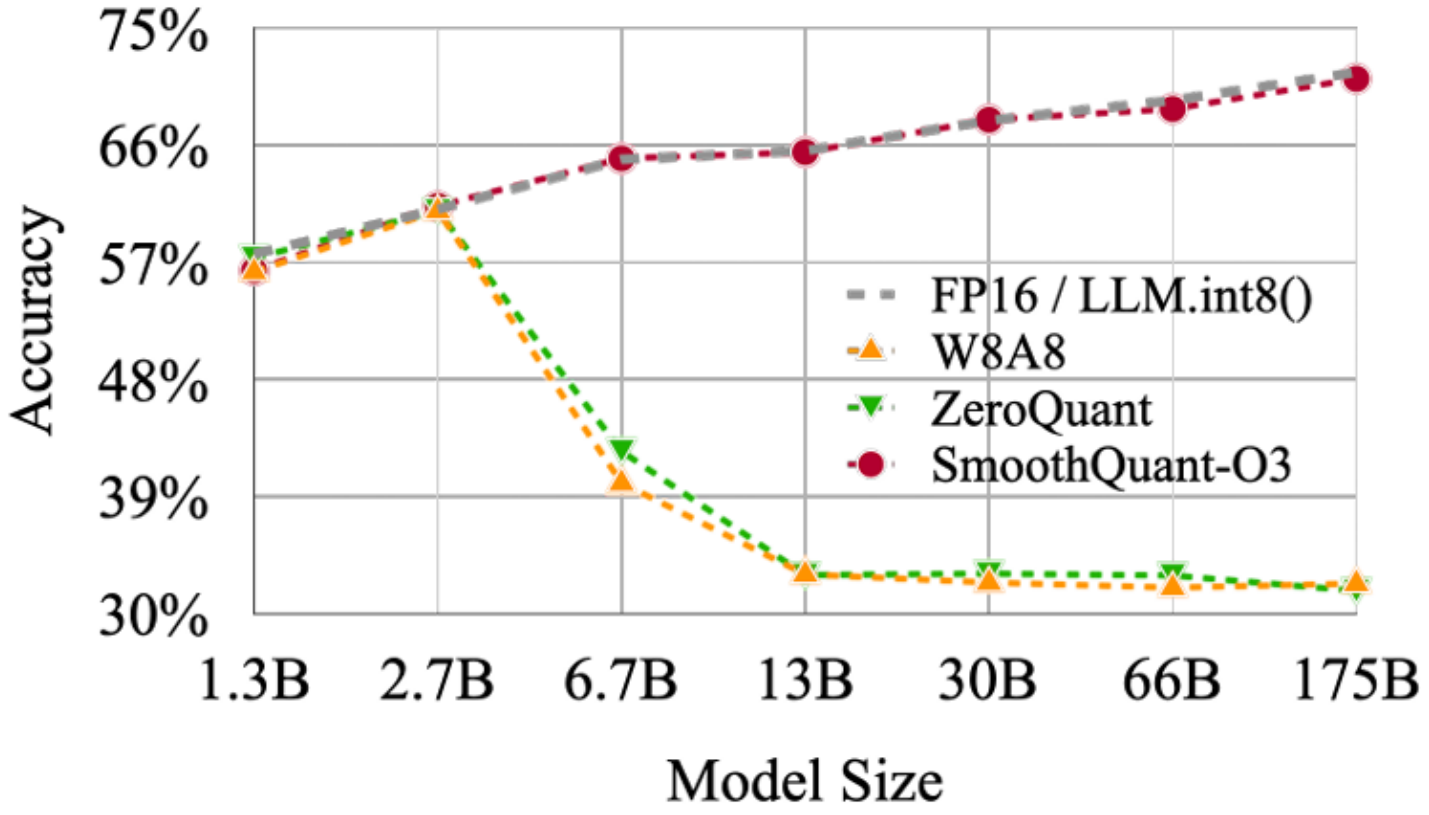


Compute-heavy GEMMs benefit from INT8 kernels, while normalization and softmax may stay higher precision.

Source: Xiao et al., SmoothQuant; mixed operator precision schematic from supplied lecture.

SmoothQuant preserves accuracy across model scales

Compare W8A8 with FP16 and naive INT8 across OPT sizes.

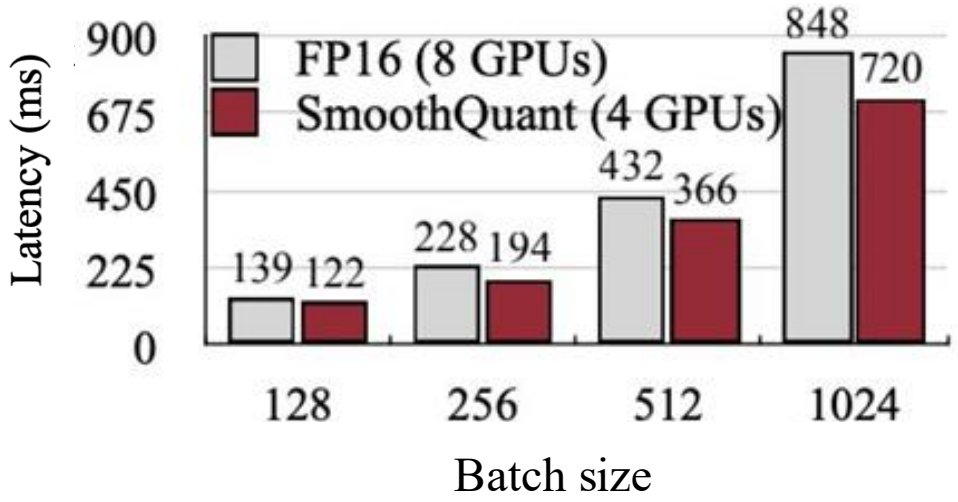


Takeaways SmoothQuant's W8A8 accuracy stays close to FP16 in the paper's evaluated OPT models.

Source: Xiao et al., SmoothQuant, ICML 2023; OPT model accuracy figure from supplied lecture.

Lower precision can change the required GPU count

Reported OPT-66B and OPT-175B performance on the evaluated GPU setup.





Takeaways W8A8 can improve serving efficiency when kernels and memory placement exploit the lower precision.

Source: Xiao et al., SmoothQuant, ICML 2023; GPU/latency comparison from supplied lecture.

Where SmoothQuant can struggle

Calibration drift	Future workloads can have different activation outliers.
Low-bit regimes	W8A8 does not automatically solve W4 / W3 accuracy.
Kernel availability	Packed formats and quantization overhead may erase theoretical speedup.
End-to-end cost	Memory savings do not directly imply lower user-visible latency.

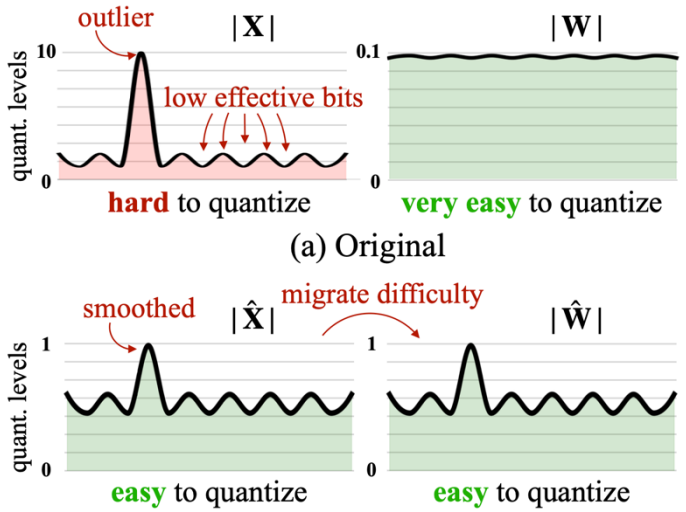
Takeaways Evaluate numerical stability, hardware support, and deployment workloads together.

Summary

Quantization Maps continuous values to fewer representable levels; clipping and rounding matter.

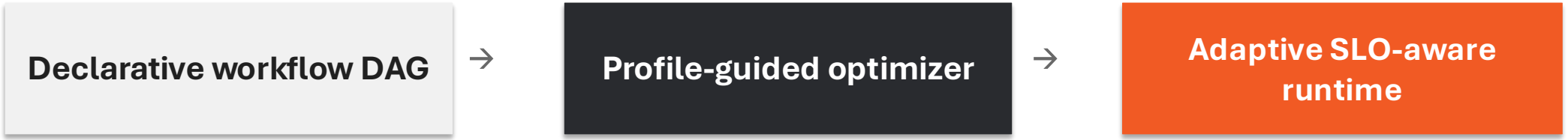
Granularity Controls whether outliers affect many values or only one group / channel.

SmoothQuant Rescales X and W equivalently to enable accurate W8A8 inference.



Panel preview: Murakkab (OSDI 2026)

Agentic workflows need joint planning across agents, models, and hardware.



Reported gains: up to 2.8× less GPU use, 3.7× less energy, and 4.3× lower cost (vs. LangGraph).

Takeaways Decouple application logic from execution choices; jointly meet quality and latency SLOs.

Source: Chaudhry et al., “Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms,” OSDI 2026.

References and acknowledgments

SmoothQuant	Xiao et al. Accurate and Efficient Post-Training Quantization for LLMs. ICML 2023.
AWQ	Lin et al. Activation-aware Weight Quantization. MLSys 2024 (Best Paper).
LLM.int8()	Dettmers et al. 8-bit Matrix Multiplication for Transformers at Scale. NeurIPS 2022.
Lecture sources	Original CS 498 L16 slide figures supplied by the instructor; thanks to Minjia Zhang, Arvind Krishnamurthy, and colleagues.

Course: CS 498 · Machine Learning Systems · UIUC